



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

TWORZENIE SYSTEMÓW ROBOTYCZNYCH

Ćwiczenia 2

ROS2 Topics

10 marca 2025



Spis treści

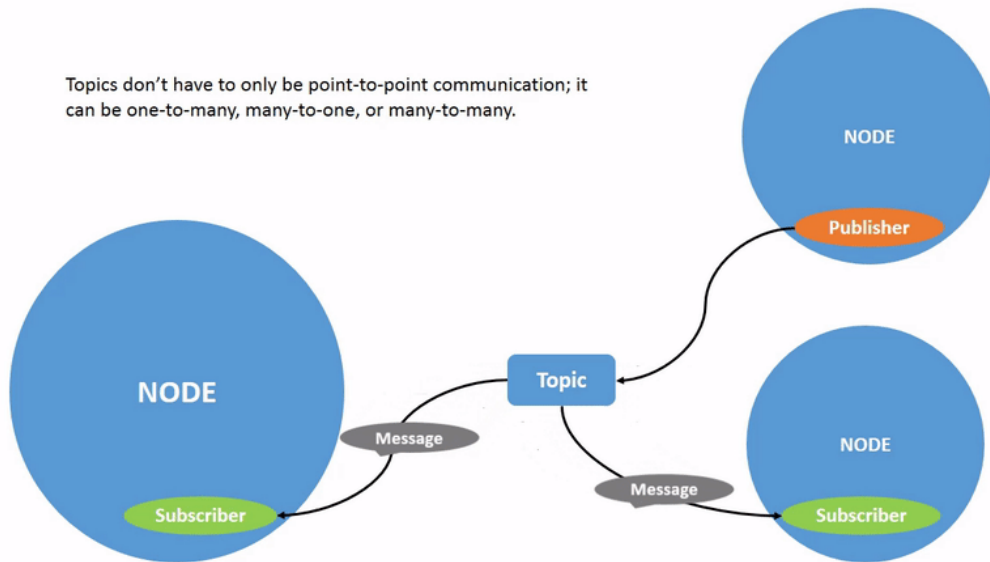
1	Wprowadzenie	2
1.1	Przykład - Radio	2
1.2	Przykład - Powiadomienia w aplikacji pogodowej	3
1.3	Inne cechy ROS Topic	3
2	Struktura wiadomości	3
2.1	Typy danych w wiadomościach	3
2.1.1	Tablice i listy	4
2.1.2	Zagnieżdżone wiadomości	4
2.1.3	Czas i czas trwania	4
2.2	Gotowe wiadomości ROS2	4
2.3	Jak sprawdzić typ wiadomości w ROS2?	4
2.4	Procedura dodania nowej wiadomości .msg do pakietu	5
2.4.1	Przejsć do pakietu <code>example_tsr_msgs</code>	5
2.4.2	Utworzenie podkatalogu <code>msg</code>	5
2.4.3	Utworzenie pliku z wiadomością .msg	5
2.4.4	Dodanie interfejsu do <code>CMakeLists.txt</code>	6
2.4.5	Modyfikacja <code>package.xml</code>	7
2.4.6	Budowanie pakietu	8
2.4.7	Weryfikacja działania	8
3	Procedura tworzenia nowego Publisher'a - brak istniejącego Subscriber'a	9
3.1	Przejsć do lokalizacji pakietu <code>example_tsr_project</code>	9
3.2	Utworzenie pliku <code>new_publisher.py</code>	9
3.3	Napisanie kodu dla węzła	9
3.4	Aktualizacja <code>setup.py</code>	11
3.5	Budowanie paczki	11
3.6	Weryfikacja działania	11
4	Procedura tworzenia Publisher'a dla istniejącego Subscriber'a	13
4.1	Przejsć do lokalizacji pakietu <code>example_tsr_project</code>	13
4.2	Utworzenie pliku <code>new_publisher_turtlesim_velocities.py</code>	13
4.3	Sprawdzenie typu wiadomości odbieranej przez istniejący Subscriber.	13
4.4	Napisanie kodu dla węzła	14
4.5	Aktualizacja <code>setup.py</code>	15
4.6	Modyfikacja <code>package.xml</code> w pakiecie <code>example_tsr_project</code>	16
4.7	Budowanie paczki	16
4.8	Weryfikacja działania	16

5	Procedura tworzenia nowego Subscribiera – brak istniejącego Publishera	17
5.1	Przejsięcie do lokalizacji pakietu <code>example_tsr_project</code>	17
5.2	Utworzenie pliku <code>new_subscriber.py</code>	18
5.3	Ustalenie typu odbieranej wiadomości	18
5.4	Napisanie kodu dla węzła	19
5.5	Aktualizacja <i><code>setup.py</code></i>	19
5.6	Budowanie paczki	20
5.7	Weryfikacja działania	20
6	Procedura tworzenia Subscribiera dla istniejącego Publishera	21
6.1	Przejsięcie do lokalizacji pakietu <code>example_tsr_project</code>	21
6.2	Utworzenie pliku <code>new_subscriber_turtlesim_pose.py</code>	21
6.3	Sprawdzenie typu wiadomości wysyłanej przez istniejący Publisher.	21
6.4	Napisanie kodu dla węzła	22
6.5	Aktualizacja <i><code>setup.py</code></i>	23
6.6	Modyfikacja <code>package.xml</code> w pakiecie <code>example_tsr_project</code>	23
6.7	Budowanie paczki	24
6.8	Weryfikacja działania	24
7	Procedura tworzenia jednoczesnego Publishera i Subscribiera	25
7.1	Przejsięcie do lokalizacji pakietu <code>example_tsr_project</code>	25
7.2	Utworzenie pliku <code>pub_sub_turtlesim.py</code>	25
7.3	Napisanie kodu dla węzła	26
7.4	Aktualizacja <i><code>setup.py</code></i>	26
7.5	Budowanie paczki	27
7.6	Weryfikacja działania	27
8	Dokumentacja i inne linki	28

1 Wprowadzenie

ROS Topic jest jednym z podstawowych sposobów wymiany informacji między węzłami (nodes). Jest to jednokierunkowa i rozgłoszeniowa forma komunikacji, co oznacza, że jeden węzeł może wysyłać dane, a inne mogą je odbierać bez konieczności nawiązywania bezpośredniego połączenia.

Komunikacja w ROS2 za pomocą Topiców opiera się na modelu **Publisher-Subscriber**.



Rys. 1.1: Komunikacja dla ROS Topic. Animacja dostępna na <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>

ROS2 Topic składa się z:

- Publisher – Wysyła wiadomości na określony temat (topic). Może to być np. stacja radiowa nadająca sygnał.
- Subscriber – Odbiera wiadomości publikowane na danym temacie. Może to być radioodbiornik, który dostraja się do danej częstotliwości i odbiera transmisję.
- Topic/Temat (kanał komunikacji) – Określa nazwę i typ przesyłanych wiadomości, np. jak częstotliwość radiowa lub grupa na WhatsAppie.

1.1 Przykład - Radio

W tym modelu każdy, kto dostroi się do odpowiedniej częstotliwości (topic), otrzyma te same dane.

- Topic (temat) - 103.5 FM
- Publisher - Stacja radiowa nadająca wiadomości
- Subscriber - Każda osoba, która włączy radio na tej częstotliwości
- Wiadomość - Muzyka lub komunikaty informacyjne

1.2 Przykład - Powiadomienia w aplikacji pogodowej

Każdy, kto ma subskrybowaną daną lokalizację, otrzymuje te same dane.

- Topic (temat) - Aktualna pogoda w danym mieście
- Publisher - Serwis pogodowy aktualizujący dane co godzinę
- Subscriber - Użytkownicy aplikacji pogodowej, którzy otrzymują powiadomienia o deszczu
- Wiadomość - „Za godzinę zacznie padać deszcz”

1.3 Inne cechy ROS Topic

- Asynchroniczność – Publisher nie musi czekać na odpowiedź, wysyła dane niezależnie od tego, czy ktoś je odbiera.
- Wielu subskrybentów – Jeden Publisher może wysyłać dane do wielu subskrybentów jednocześnie.
- Brak bezpośredniego połączenia – Publisher nie „wie”, kto go słucha, wysyła dane na określony temat i każdy zainteresowany może je odebrać.
- Luźne powiązanie – Subskrybenci mogą pojawiać się i znikać, a Publisher nadal działa bez problemu.

2 Struktura wiadomości

Dla komunikacji ROS2 Topic dane wysyłane są na unikalny temat pomiędzy węzłami w formie wiadomości o określonej strukturze. Każda wiadomość posiada predefiniowane pola i typy danych, zdefiniowane w plikach `.msg`.

Format wiadomości (.msg)

Plik `.msg` składa się z kolejnych linii, gdzie każda linia określa:

- **Typ danych** (np. `int32`, `float64`, `string`),
- **Nazwę pola** (np. `id`, `velocity`, `temperature`).

Przykładowa wiadomość:

```
1 int32 sensor_id
2 float64 temperature
3 string status
```

2.1 Typy danych w wiadomościach

Szczegółowe informacje dotyczące dokumentacji i typów pól znajduje się:

<https://docs.ros.org/en/foxy/Concepts/About-ROS-Interfaces.html>

ROS2 obsługuje następujące typy danych:

Podstawowe typy

- `bool` – wartość logiczna (prawda/fałsz),
- `int8`, `int16`, `int32`, `int64` – liczby całkowite ze znakiem,
- `uint8`, `uint16`, `uint32`, `uint64` – liczby całkowite bez znaku,

- `float32`, `float64` – liczby zmiennoprzecinkowe,
- `string` – tekst,
- `wstring` – tekst w Unicode.

2.1.1 Tablice i listy

Można tworzyć dynamiczne i statyczne listy:

```
1 int32[] measurements      # Dowolna liczba pomiarów
2 float64[3] position      # Stała tablica (x, y, z)
3 string[<=5] error_codes   # Maksymalnie 5 kodów błędów
```

2.1.2 Zagnieżdżone wiadomości

Wiadomości mogą zawierać inne wiadomości. W tym przykładzie zakładamy, że obie wiadomości znajdują się w tym samym pakiecie.

```
1 # Plik Position.msg
2 float64 x
3 float64 y
4 float64 z
```

```
1 # Plik RobotState.msg
2 string robot_name
3 Position position
4 bool is_active
```

2.1.3 Czas i czas trwania

ROS2 obsługuje typy czasowe:

```
1 builtin_interfaces/Time timestamp
2 builtin_interfaces/Duration duration
```

2.2 Gotowe wiadomości ROS2

ROS2 posiada wiele typów wiadomości, które są dostępne w różnych pakietach np.:

- `std_msgs` – `String`, `Int32`, `Float64`, `Bool`,
- `geometry_msgs` – `Pose`, `Twist`, `Vector3`,
- `sensor_msgs` – `LaserScan`, `Image`, `Imu`,
- `nav_msgs` – `Odometry`, `Path`.

Przykład wiadomości:

```
1 geometry_msgs/Point position
2 geometry_msgs/Quaternion orientation
```

2.3 Jak sprawdzić typ wiadomości w ROS2?

W terminalu można zobaczyć wiadomości używane w systemie - lista dostępnych tematów:

```
1 ros2 topic list
```

Sprawdzenie typu wiadomości na temacie `/example_topic`:

```
1 ros2 topic type /example_topic
2 # lub alternatywna wersja wyświetlająca więcej informacji o samym temacie wraz z typem
3 ros2 topic info /example_topic
```

Dla otrzymanego typu sprawdzić strukturę wiadomości typu `String` z pakietu `std_msgs`. Podana wartość jako ostatni argument jest wynikiem otrzymanego wcześniej typu.

```
1 ros2 interface show std_msgs/msg/String
```

Przykładowy wynik:

```
1 string data
```

2.4 Procedura dodania nowej wiadomości `.msg` do pakietu

Jest to przypomnienie dodanej na poprzednich zajęciach. Podane pliki i katalogi istnieją już w tym pakiecie `example_tsr_msgs`. Procedura natomiast zostanie tak opisana jakby ich nie było. Zakładamy, że dodajemy wiadomości do już istniejącego pakietu, który może nie być skonfigurowany do generowania nowych wiadomości. Może brakować folderów.

2.4.1 Przejść do pakietu `example_tsr_msgs`

Należy w terminalu przejść do workspace'a (`example_tsr_ws`) a następnie do `src` i pakietu `example_tsr_msgs`:

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs
```

2.4.2 Utworzenie podkatalogu `msg`

Utworzenie w pakiecie `example_tsr_msgs` katalogów dla interfejsów komunikacyjnych dla ROS Topic (katalog `msg`) W tym celu należy przejść do lokalizacji głównego katalogu dla pakietu a następnie utworzyć katalog `msg`. Można dodać ręcznie katalog z pominięciem terminala.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs
2 mkdir msg
```

2.4.3 Utworzenie pliku z wiadomością `.msg`

W katalogu `msg` (`~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs/msg`) utworzyć plik tekstowy `ExampleMsgType.msg`, w którym zdefiniowany zostanie interfejs komunikacyjny dla wiadomości ROS Topic. Wszystkie pliki w tym katalogu będą miały rozszerzenie `.msg`. Do pliku należy wkleić przykład struktury wiadomości:

```
1 int32 id
2 string name
3 float32 value
4 bool is_valid
5 int32[] dynamic_integer_array
6 geometry_msgs/Point example_goal
```

Z lewej strony znajduje się typ pola wiadomości a z prawej nazwa pola, która jest wykorzystywana przy uzupełnianiu lub odczycie wartości z pola w wiadomości. W tym pliku znajduje się kilka przykładowych typów pól wiadomości. Pola zawierające listę elementów oznaczone są []. W tym przykładzie lista z wartościami typu `int_32` przypisana jest do pola `dynamic_integer_array`.

Oprócz podstawowych typów można używać bardziej zaawansowanych struktur pochodzących z innych pakietów. W tej wiadomości `example_goal` odnosi się do struktury wiadomości zdefiniowanej w pakiecie `geometry_msgs`.



```

Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show geometry_msgs/msg/Point
# This contains the position of a point in free space
float64 x
float64 y
float64 z
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$

```

Rys. 2.1: Struktura wiadomości `geometry_msgs/Point`

2.4.4 Dodanie interfejsu do `CMakeLists.txt`.

W głównym katalogu pakietu `example_tsr_ws_msgs` znajduje się plik `CMakeLists.txt`, który należy otworzyć w celu edycji. Do pliku należy dodać następujące linie:

```

1 find_package(geometry_msgs REQUIRED)
2
3 find_package(rosidl_default_generators REQUIRED)
4
5 rosidl_generate_interfaces(${PROJECT_NAME}
6   "msg/ExampleMsgType.msg"
7   DEPENDENCIES geometry_msgs
8 )
9
10 ament_export_dependencies(rosidl_default_runtime)
11

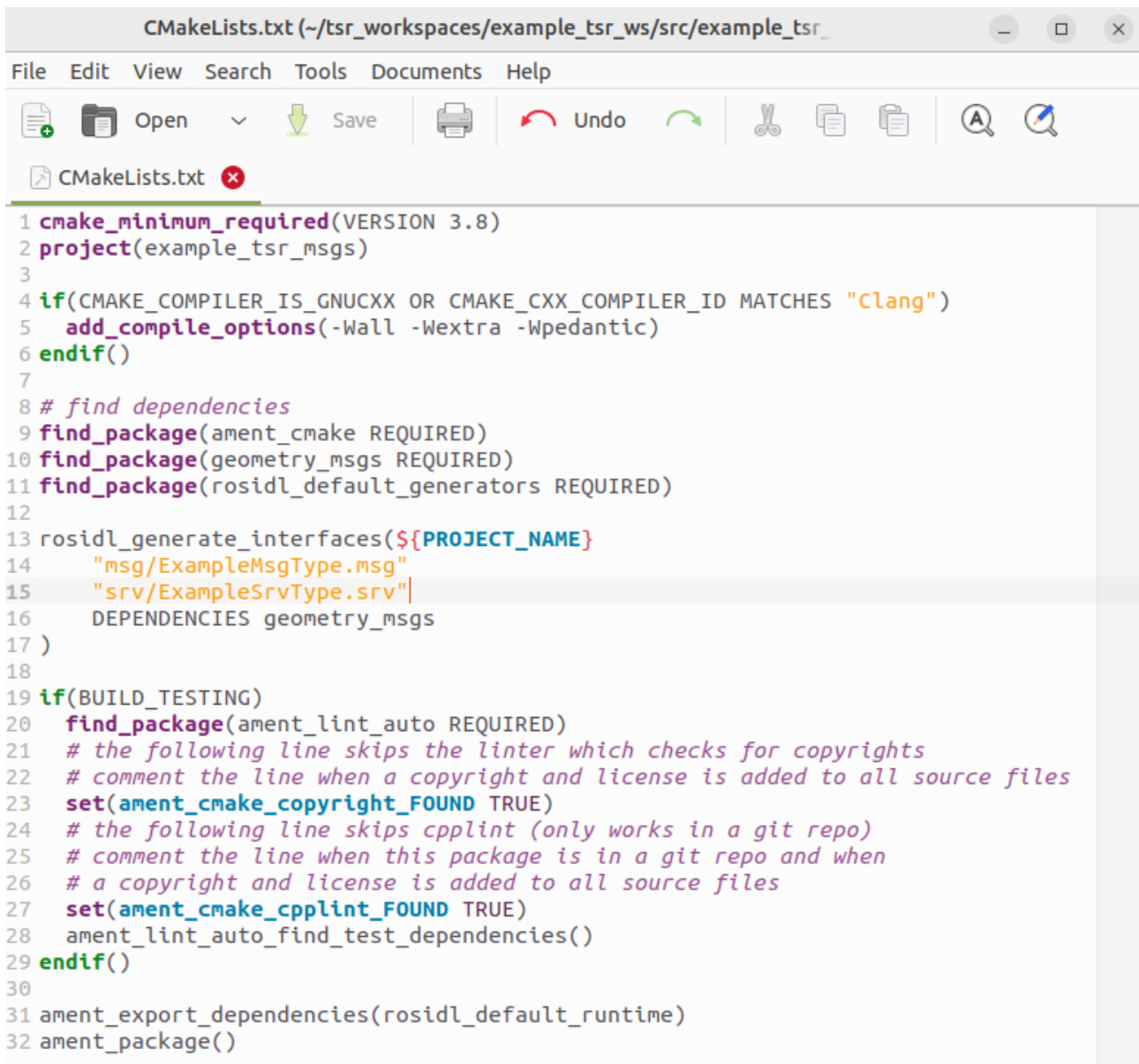
```

Linia 1 odpowiada za znalezienie wymaganych zależności od innych pakietów. W przykładowej wiadomości `.msg` wykorzystana została wiadomość typu `Point` z paczki `geometry_msgs` z tego powodu ta zależność pojawia się w `CMakeLists.txt`. Zależność pojawiająca się w wiadomości od innej paczki należy również dodać do `rosidl_generate_interfaces` za `DEPENDENCIES` (linia 8).

Do wygenerowania nowych interfejsów komunikacyjnych niezbędne jest zaimportowanie pakietu `rosidl_default_generators` (linia 3).

W części `rosidl_generate_interfaces` (linie 5-9) należy dodać utworzone interfejsy komunikacyjne oraz zależności z nimi związane. Należy również wyeksportować zależności od `message runtime` (linia 11)

Po edycji plik powinien wyglądać następująco (Rys. 2.2):



Rys. 2.2: Plik CMakeLists.txt po edycji dla paczki example_tsr_msgs

2.4.5 Modyfikacja package.xml

Następnie należy dokonać edycji pliku package.xml i dodać następujące zależności, jeśli nie są dodane.

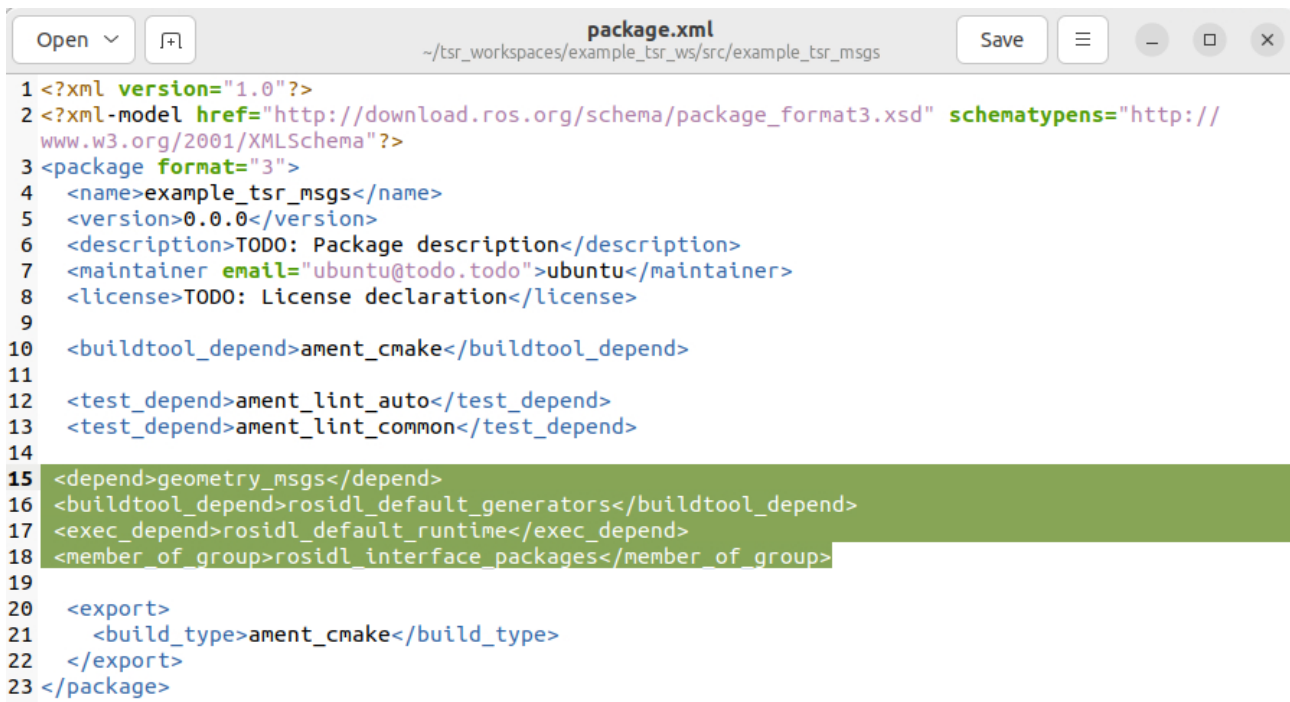
```

1 <depend>geometry_msgs</depend>
2 <buildtool_depend>rosidl_default_generators</buildtool_depend>
3 <exec_depend>rosidl_default_runtime</exec_depend>
4 <member_of_group>rosidl_interface_packages</member_of_group>

```

Linie 1 należy dodać, gdyż korzystamy z wiadomości typu `Point` z pakietu `geometry_msgs` w tym pakiecie (`example_tsr_msgs`). Tak samo należy dodać zależności od innych pakietów. Linia 2 określa narzędzie budowania potrzebne do generowania kodu dla interfejsów komunikacyjnych (`msg`, `srv`, `action`). Linia 3 jest niezbędna do późniejszego wykorzystania interfejsów. `rosidl_interface_packages` jest nazwą grupy z zależnościami w utworzonym pakiecie.

Po edycji plik powinien wyglądać następująco (Rys. 2.3):



Rys. 2.3: Plik package.xml po edycji dla pakietu example_tsr_msgs

2.4.6 Budowanie pakietu

Należy przejść do głównego katalogu dla workspace'a (example_tsr_ws):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczy pakiet (można też zbudować cały workspace colcon build):

```
1 colcon build --packages-select example_tsr_msgs
```

i wykonać ponowny source w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

2.4.7 Weryfikacja działania

Dla utworzonego interfejsu dla ROS Topic należy sprawdzić czy nowo utworzony interfejs jest widoczny.

```
1 ros2 interface show example_tsr_msgs/msg/ExampleMsgType
```

Prawidłowy rezultat przedstawia (Rys. 2.4):

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 interface show example_tsr_msgs/msg/ExampleMsgType
int32 id
string name
float32 value
bool is_valid
int32[] dynamic_integer_array
geometry_msgs/Point example_goal
  float64 x
  float64 y
  float64 z
```

Rys. 2.4: Wynik operacji sprawdzenia w terminalu struktury utworzonych interfejsów dla pakietu example_tsr_msgs.

Po użyciu polecenia sprawdzającego strukturę interfejsu nie mogą pojawić się błędy. Struktura ma być zgodna z tą jaka jest w dodanym pliku `ExampleMsgType.msg`. Jeśli wyświetla się coś innego tzn., że należy ponownie zweryfikować poprawność konfiguracji i przejść procedurę jeszcze raz zwracając dokładniejszą uwagę na wykonywane kroki.

3 Procedura tworzenia nowego Publishera - brak istniejącego Subscribiera

3.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy Publisher. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

3.2 Utworzenie pliku `new_publisher.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik *`new_publisher.py`* w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/new_publisher.py
2 chmod +x example_tsr_project/new_publisher.py
```

Nowo utworzony Publisher powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/new_publisher.py
```

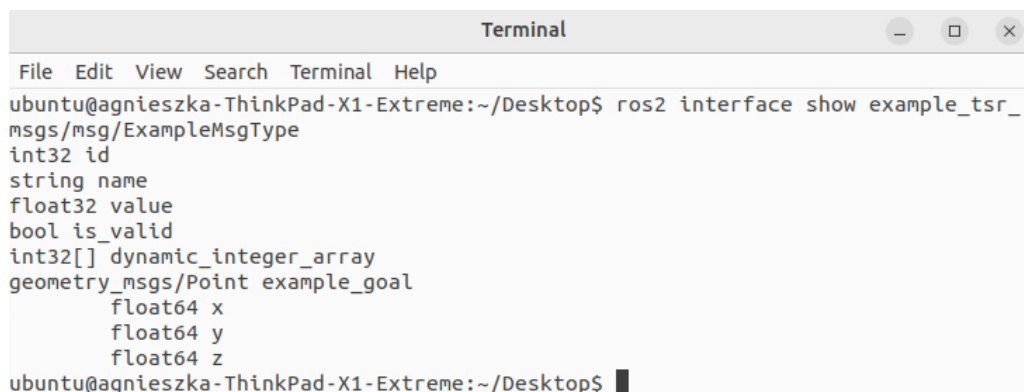
3.3 Napisanie kodu dla węzła

Następnie należy utworzyć nowy węzeł z Publisherem wysyłającym na temacie `/new_unique_topic_name` wiadomość typu `ExampleMsgType` z utworzonego pakietu `example_tsr_msgs`. Do utworzenia Publishera niezbędna jest znajomość struktury tej wiadomości, którą można sprawdzić w terminalu w następujący sposób:

```
1 ros2 interface show example_tsr_msgs/msg/ExampleMsgType
```

W tym miejscu można wybrać dowolny inny typ wiadomości a później konsekwentnie go stosować w odpowiednich miejscach. Przykładowo dla innego typu wiadomości np. PoseStamped z pakietu geometry_msgs w poniższych miejscach dla opisanej procedury należy zamiast ExampleMsgType wstawić PoseStamped a zamiast example_tsr_msgs powinno pojawić się geometry_msgs. **Należy pamiętać, że w innym typie wiadomości pola będą miały inną nazwę.**

W wyniku operacji wyświetla się struktura wiadomości dla ROS Topic (Rys. 3.1):



Rys. 3.1: Struktura wiadomości ExampleMsgType

Znajomość typów i pól w wiadomości jest niezbędna do jej prawidłowego uzupełnienia.

Następnie należy napisać prostego Publishera (**zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu**):

```

1 import rclpy # import biblioteki do obsługi ROS2
2 from rclpy.node import Node
3 from example_tsr_msgs.msg import ExampleMsgType # Import wiadomości ROS2 typu ExampleMsgType z pakietu
  ↳ example_tsr_msgs
4
5 class SimplePublisher(Node): # Utworzenie klasy dla Node'a
6     def __init__(self):
7         super().__init__('simple_publisher') # unikalna nazwa node'a
8
9         # Utworzenie atrybutu klasy będącego Publisherem wiadomości typu ExampleMsgType na temacie
10        ↳ /new_unique_topic_name.
11        self.publisher_ = self.create_publisher(ExampleMsgType, '/new_unique_topic_name', 10)
12
13        # Utworzenie timera, co 1s wywołuje się metoda self.timer_callback
14        self.timer = self.create_timer(1.0, self.timer_callback)
15
16    def timer_callback(self):
17        msg = ExampleMsgType() # Utworzenie wiadomości ROS2 typu ExampleMsgType
18
19        # Uzupełnienie pól wiadomości
20        msg.id = 123
21        msg.name = "Nowa wiadomosc"
22        msg.is_valid = True
23        msg.dynamic_integer_array = [3, 4, 5]
24
25        # Wysłanie wiadomości
26        self.publisher_.publish(msg)
27
28        # Wysłanie informacji do logów.
29        self.get_logger().info(f'Opublikowano: {msg}')
30
31 def main(args=None):
32     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
33     rclpy.init(args=args)
34
35     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy SimplePublisher
36     node = SimplePublisher()
37
38     # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
39     ↳ zakończenie działania programu.
40     rclpy.spin(node)
41
42     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
43     node.destroy_node()
44
45     # Zamyka połączenie z ROS2.
46     rclpy.shutdown()
47
48 if __name__ == '__main__':

```

```
47     # uruchomienie node'a
48     main()
```

3.4 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za publikowanie utworzonej przed chwilą wiadomości. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```
1  entry_points={
2      'console_scripts': [
3          'new_talker_node_name = example_tsr_project.new_publisher:main',
4      ],
5  }
```

Lewa strona w linii z dodanym `new_talker_node_name` odnosi się do unikalnej nazwy w pakiecie, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa nie będzie widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`). Zamiast tego będzie widoczna nazwa wynikająca z konfiguracji węzła w pliku z programem. Wpisana w konstruktorze tam gdzie pojawia się `super().__init__('simple_publisher')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `simple_publisher`.

Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `new_publisher.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

3.5 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1  cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1  colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

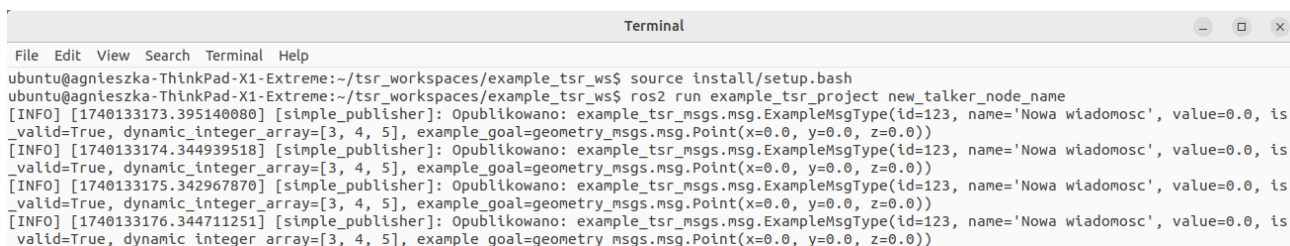
```
1  source install/setup.bash
```

3.6 Weryfikacja działania

W terminalu należy wpisać:

```
1  ros2 run example_tsr_project new_talker_node_name
```

W wyniku (Rys. 3.2) działania polecenia publikowana jest wiadomość na temacie `/new_unique_topic_name` typu `ExampleMsgType`. Ponadto w terminalu wyświetla się informacja wysyłana do logów.



```

Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ source install/setup.bash
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 run example_tsr_project new_talker_node_name
[INFO] [1740133173.395140080] [simple_publisher]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is
_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740133174.344939518] [simple_publisher]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is
_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740133175.342967870] [simple_publisher]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is
_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740133176.344711251] [simple_publisher]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is
_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))

```

Rys. 3.2: Wynik uruchomienia nowego węzła `new_talker_node_name`

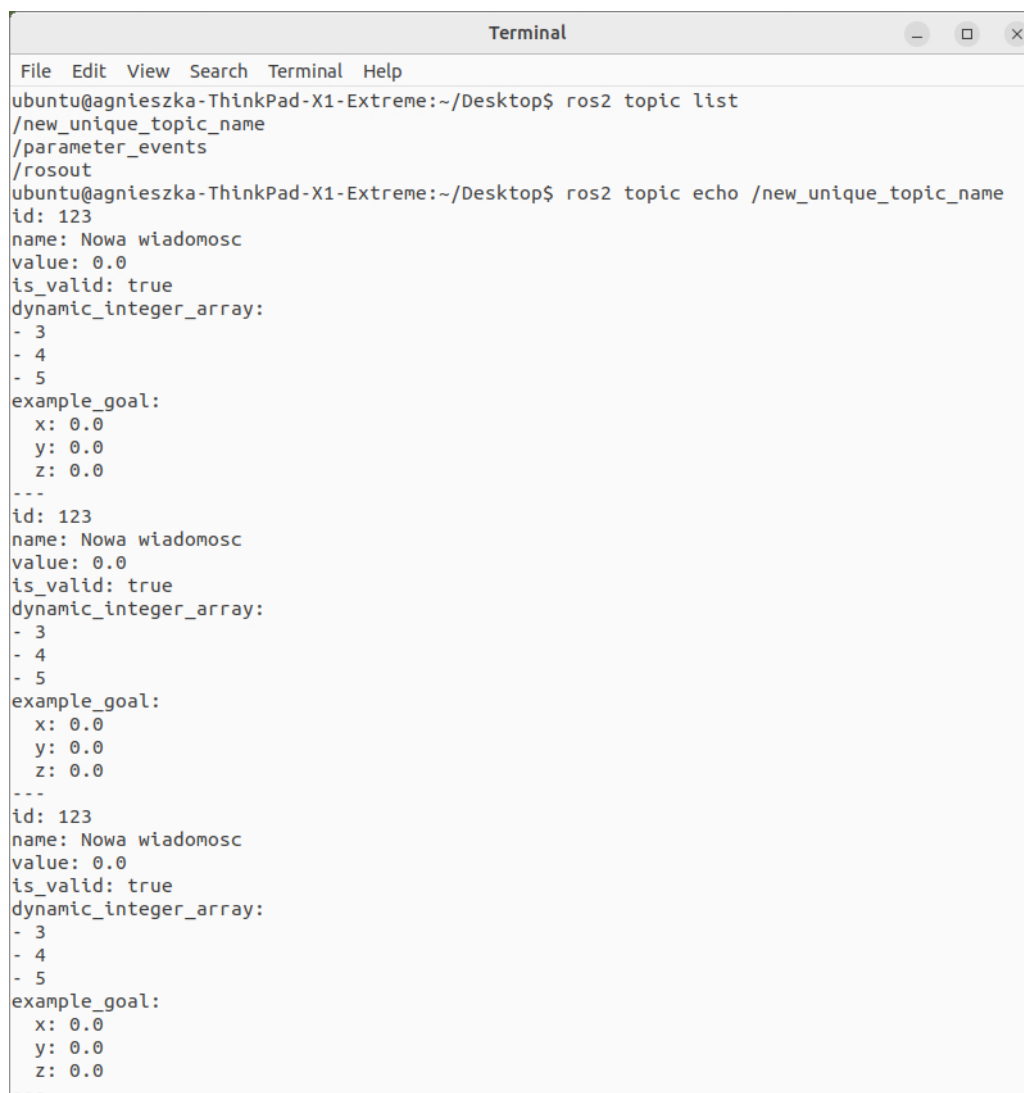
W nowym terminalu można sprawdzić listę aktywnych tematów (topic'ów) i na liście powinien pojawić się `/new_unique_topic_name`:

```
1 ros2 topic list
```

a następnie komendę umożliwiającą podgląd wiadomości wysyłanych na temacie `/new_unique_topic_name`.

```
1 ros2 topic echo /new_unique_topic_name
```

W wyniku wpisanych komend w terminalu powinien być wyświetlony następujący rezultat (Rys. 3.3), w którym cyklicznie wysyłana jest ta sama wiadomość (rozdzielenie wiadomości — — —):



```

Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 topic list
/new_unique_topic_name
/parameter_events
/rosout
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 topic echo /new_unique_topic_name
id: 123
name: Nowa wiadomosc
value: 0.0
is_valid: true
dynamic_integer_array:
- 3
- 4
- 5
example_goal:
  x: 0.0
  y: 0.0
  z: 0.0
---
id: 123
name: Nowa wiadomosc
value: 0.0
is_valid: true
dynamic_integer_array:
- 3
- 4
- 5
example_goal:
  x: 0.0
  y: 0.0
  z: 0.0
---
id: 123
name: Nowa wiadomosc
value: 0.0
is_valid: true
dynamic_integer_array:
- 3
- 4
- 5
example_goal:
  x: 0.0
  y: 0.0
  z: 0.0
---

```

Rys. 3.3: Sprawdzenie poprawności działania węzła `simple_publisher`

4 Procedura tworzenia Publishera dla istniejącego Subscribiera

4.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy Publisher. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

4.2 Utworzenie pliku `new_publisher_turtlesim_velocities.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik *`new_publisher_turtlesim_velocities.py`* w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/new_publisher_turtlesim_velocities.py
2 chmod +x example_tsr_project/new_publisher_turtlesim_velocities.py
```

Nowo utworzony Publisher powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/new_publisher_turtlesim_velocities.py
```

4.3 Sprawdzenie typu wiadomości odbieranej przez istniejący Subscriber.

W tym celu na początku należy uruchomić w nowym terminalu symulację dla robotów żółwi (`turtlesim`):

```
1 ros2 run turtlesim turtlesim_node
```

Następnie w **nowym terminalu** wyświetlamy listę tematów obsługiwanych przez symulację:

```
1 ros2 topic list
```

W symulacji dla każdego robota dostępne są 3 tematy:

- `/turtle1/cmd_vel` - do wysyłania prędkości sterujących robotem.
- `/turtle1/color_sensor` - do odczytu aktualnego koloru na jakim znajduje się robot. Jeśli ma ustawiony pisak to jest to cały czas kolor pisaka.
- `/turtle1/pose` - do odczytu aktualnego położenia robota.

`turtle1` odnosi się do nazwy utworzonego robota, jeśli będzie ich więcej to tematy będą się między sobą różniły się tylko tą nazwą. Pozostała część pozostanie bez zmian. Ponadto każda wiadomość będzie tego samego typu. Różnica jest w przesyłanych danych (wiadomość jest uzupełniona innymi wartościami).

Dla tematu od prędkości `cmd_vel` dla węzła symulacji utworzony jest Subscriber i dla niego zostanie napisany nowy węzeł (node) zawierający Publisher odpowiedzialny za wysyłanie prędkości. W wyniku działania robot będzie poruszał się w oknie symulacji.

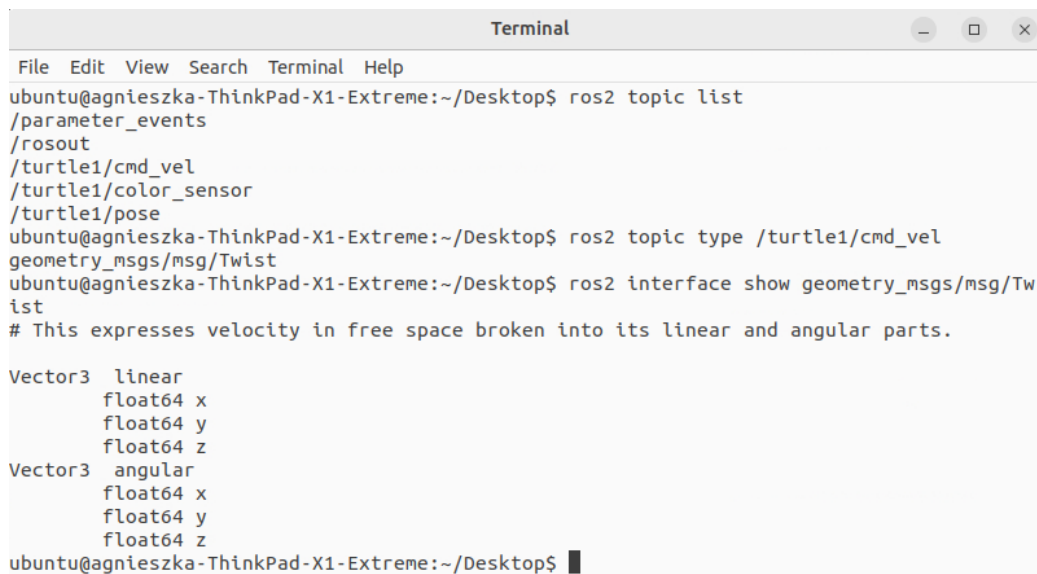
Napisanie Publisher'a dla istniejącego Subscriber'a wymaga znajomości tematu, na którym odbierane są wiadomości oraz ich typ. Znając już temat `/turtle1/cmd_vel`, na którym odbierane są wiadomości należy sprawdzić typ danych.

```
1 ros2 topic type /turtle1/cmd_vel
```

a następnie sprawdzić strukturę wiadomości, dla wyświetlonego typu:

```
1 ros2 interface show geometry_msgs/Twist
```

Wynik działania kolejnych poleceń przedstawia (Rys. 4.1):



```

Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 topic type /turtle1/cmd_vel
geometry_msgs/msg/Twist
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$

```

Rys. 4.1: Sprawdzenie typu i struktury wiadomości dla istniejącego Subscriber'a dla tematu `/turtle1/cmd_vel`

4.4 Napisanie kodu dla węzła

Następnie należy utworzyć nowy węzeł z Publisherem wysyłającym na temacie `/turtle1/cmd_vel` wiadomość typu `Twist` z pakietu `geometry_msgs`.

Następnie należy napisać Publisher'a (**zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu**):

```

1 import rclpy # import biblioteki do obsługi ROS2
2 from rclpy.node import Node
3 from geometry_msgs.msg import Twist # Import wiadomości ROS2 typu Twist z pakietu geometry_msgs
4
5 class VelTurtlePublisher(Node): # Utworzenie klasy dla Node'a
6     def __init__(self):
7         super().__init__('turtlesim_cmd_vel_publisher') # unikalna nazwa node'a. Jest to nazwa widoczna na
            ↳ liście ros2 node list. Uruchomienie wynikające z konfiguracji może mieć inną nazwę.
8
9         # Utworzenie atrybutu klasy będącego Publisherem wiadomości typu Twist na temacie /turtle1/cmd_vel.
10        self.vel_publisher_ = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
11
12        # Utworzenie timera, co 0.2s wywołuje się metoda self.timer_callback
13        self.timer = self.create_timer(0.2, self.timer_callback)
14
15    def timer_callback(self):
16        msg = Twist() # Utworzenie wiadomości ROS2 typu Twist
17
18        # Uzupełnienie pól wiadomości
19        msg.linear.x = 1.0
20        msg.angular.z = 1.0
21
22        # Wystanie wiadomości
23        self.vel_publisher_.publish(msg)
24
25
26 def main(args=None):
27     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
28     rclpy.init(args=args)
29
30     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy VelTurtlePublisher
31     node = VelTurtlePublisher()
32
33     # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
            ↳ zakończenie działania programu.
34     rclpy.spin(node)
35
36     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
37     node.destroy_node()
38
39     # Zamyka połączenie z ROS2.
40     rclpy.shutdown()
41
42 if __name__ == '__main__':
43     # uruchomienie node'a
44     main()

```

4.5 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za publikowanie utworzonej przed chwilą wiadomości. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```

1 entry_points={
2     'console_scripts': [
3         'turtle1_cmd_vel_pub = example_tsr_project.new_publisher_turtlesim_velocities:main',
4     ],
5 }

```

Lewa strona w linii z dodanym **turtle1_cmd_vel_pub** odnosi się do unikalnej nazwy, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa nie będzie widoczna po uruchomieniu węzła na liście uruchomionych węzłów (*ros2 node list*). Zamiast tego będzie widoczna nazwa wynikająca z konfiguracji węzła w pliku z programem. Wpisana w konstruktorze tam gdzie pojawia się `super().__init__('turtlesim_cmd_vel_publisher')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `turtlesim_cmd_vel_publisher`.

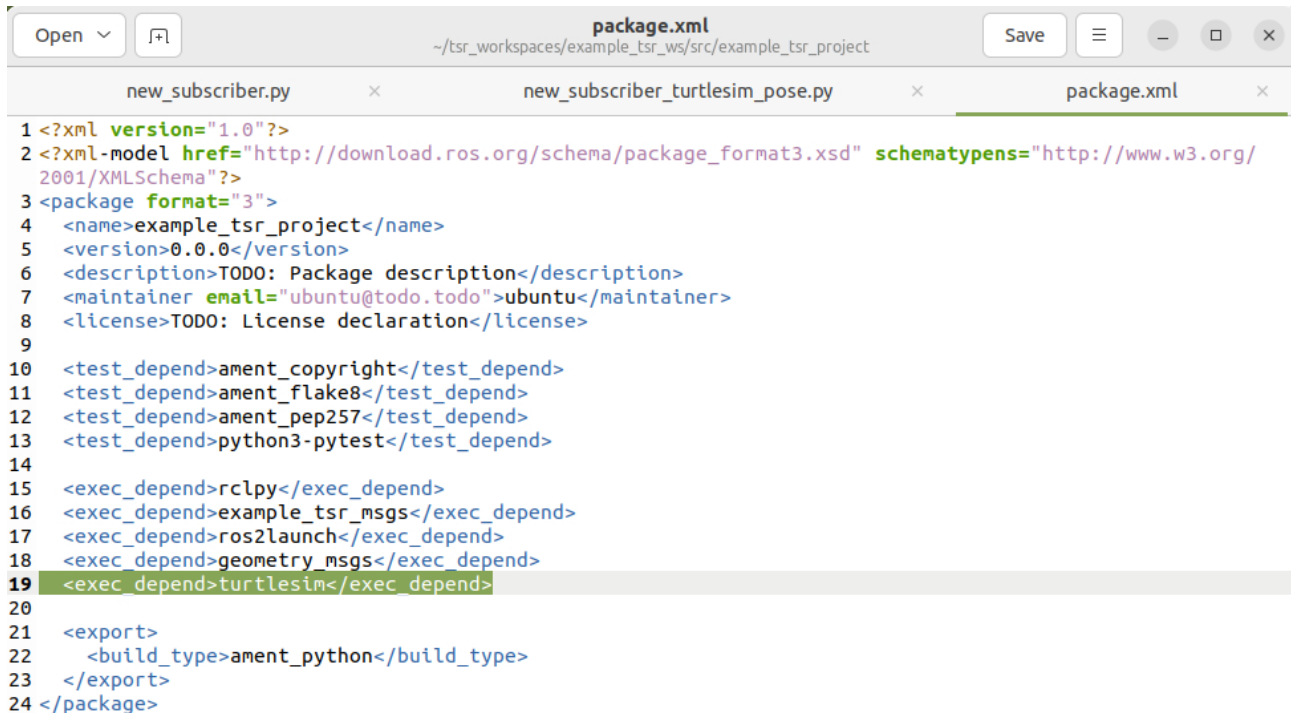
Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `new_publisher_turtlesim_velocities.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

4.6 Modyfikacja package.xml w pakiecie example_tsr_project.

Należy dodać następujące zależności do pliku package.xml:

```
1 <exec_depend>geometry_msgs</exec_depend>
```

Dodanie linii do pliku package.xml (Rys. 4.2):



Rys. 4.2: Plik package.xml po dodaniu zależności od geometry_msgs

4.7 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (example_tsr_ws):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny source w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

4.8 Weryfikacja działania

W terminalu należy wpisać:

```
1 ros2 run example_tsr_project turtle1_cmd_vel_pub
```

W wyniku uruchomienia węzła publikowana jest wiadomość na temacie `/turtle1/cmd_vel` typu `Twist`. Żółw powinien poruszać się, jeśli jedna z prędkości postępowe `x` lub obrotowa `z` jest niezerowa. Ponadto w terminalu można sprawdzić czy na temacie `/turtle1/cmd_vel` pojawiają się nowe wiadomości.

```
1  ros2 topic echo /turtle1/cmd_vel
```

W wyniku wpisanych komend w terminalu powinien być wyświetlony następujący rezultat (Rys. 4.3), w którym cyklicznie wysyłana jest ta sama wiadomość (rozdzielenie wiadomości – – –):



Rys. 4.3: Sprawdzenie poprawności działania węzła `turtlesim_cmd_vel_publisher`

5 Procedura tworzenia nowego Subscribiera – brak istniejącego Publishera

5.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy Subscriber. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1  cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

5.2 Utworzenie pliku `new_subscriber.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik *`new_subscriber.py`* w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/new_subscriber.py
2 chmod +x example_tsr_project/new_subscriber.py
```

Nowo utworzony Subscriber powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/new_subscriber.py
```

5.3 Ustalenie typu odbieranej wiadomości

Następnie należy utworzyć nowy węzeł z Subscribierem odbierającym wiadomość na temacie `/new_unique_topic_name` typu `ExampleMsgType` z utworzonego pakietu `example_tsr_msgs`. Do utworzenia Subscribiera niezbędna jest znajomość struktury tej wiadomości, którą można sprawdzić w terminalu w następujący sposób:

```
1 ros2 interface show example_tsr_msgs/msg/ExampleMsgType
```

W tym miejscu można wybrać dowolny inny typ wiadomości a później konsekwentnie go stosować w odpowiednich miejscach. Przykładowo dla innego typu wiadomości np. `PoseStamped` z pakietu `geometry_msgs` w poniższych miejscach dla opisanej procedury należy zamiast `ExampleMsgType` wstawić `PoseStamped` a zamiast `example_tsr_msgs` powinno pojawić się `geometry_msgs`. Należy pamiętać, że w innym typie wiadomości pola będą miały inną nazwę.

W wyniku operacji wyświetla się struktura wiadomości dla ROS Topic (Rys. 5.1):

```
Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_
msgs/msg/ExampleMsgType
int32 id
string name
float32 value
bool is_valid
int32[] dynamic_integer_array
geometry_msgs/Point example_goal
    float64 x
    float64 y
    float64 z
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$
```

Rys. 5.1: Struktura wiadomości `ExampleMsgType`

Znajomość typów i pól w wiadomości jest niezbędna do jej prawidłowego uzupełnienia.

5.4 Napisanie kodu dla węzła

Następnie należy napisać prostego Subscribiera (zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu):

```

1  import rclpy # import biblioteki do obsługi ROS2
2  from rclpy.node import Node
3  from example_tsr_msgs.msg import ExampleMsgType # Import wiadomości ROS2 typu ExampleMsgType z pakietu
   ↪ example_tsr_msgs
4
5  class SimpleSubscriber(Node): # Utworzenie klasy dla Node'a
6      def __init__(self):
7          super().__init__('simple_subscriber') # unikalna nazwa node'a
8
9          # Utworzenie atrybutu klasy będącego Subscribierem wiadomości typu ExampleMsgType na temacie
   ↪ /new_unique_topic_name.
10         self.subscription = self.create_subscription(ExampleMsgType, '/new_unique_topic_name', self.callback,
   ↪ 10)
11
12         def callback(self, msg):
13             self.get_logger().info(f'Id wiadomości: {msg.id} \nOdebrano: {msg}')
14
15
16     def main(args=None):
17         # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
18         rclpy.init(args=args)
19
20         # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy SimpleSubscriber
21         node = SimpleSubscriber()
22
23         # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
   ↪ zakończenie działania programu.
24         rclpy.spin(node)
25
26         # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
27         node.destroy_node()
28
29         # Zamyka połączenie z ROS2.
30         rclpy.shutdown()
31
32     if __name__ == '__main__':
33         # uruchomienie node'a
34         main()

```

5.5 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za odbieranie utworzonej przed chwilą wiadomości. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```

1  entry_points={
2      'console_scripts': [
3          'simple_subscriber = example_tsr_project.new_subscriber:main',
4      ],
5  }

```

Lewa strona w linii z dodanym **simple_subscriber** odnosi się do unikalnej nazwy w pakiecie, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa nie będzie widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`). Zamiast tego będzie widoczna nazwa wynikająca z konfiguracji węzła w pliku z programem. Wpisana w konstruktorze tam gdzie pojawia się `super().__init__('simple_subscriber')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `simple_subscriber`. Jeśli ten sam węzeł nie jest uruchamiany kilka razy z różnymi parametrami to może mieć taką samą nazwę jak wynika z konfiguracji w pliku, aby łatwiej go było znaleźć.

Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `new_subscriber.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

5.6 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

5.7 Weryfikacja działania

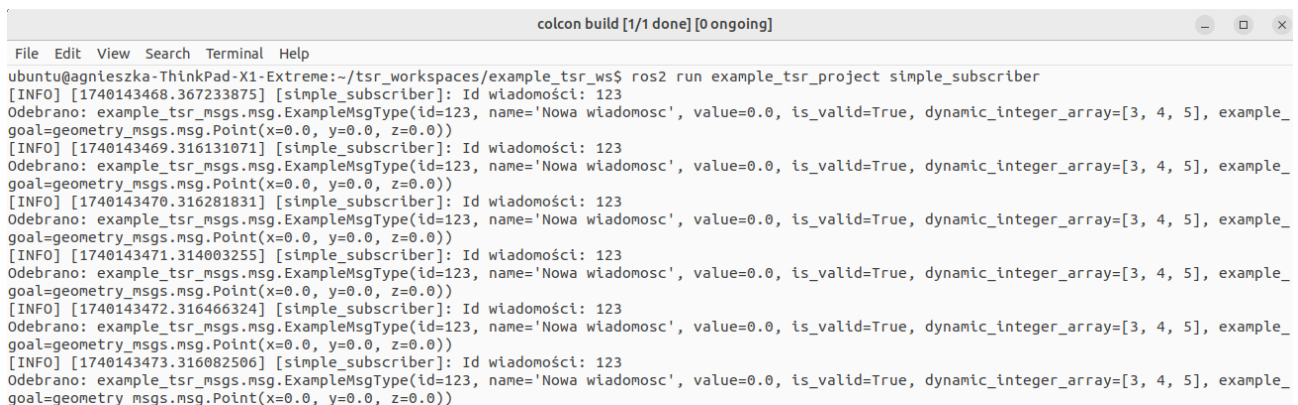
W terminalu należy wpisać:

```
1 ros2 run example_tsr_project simple_subscriber
```

Teraz węzeł z Subscriberem oczekuje na otrzymanie informacji na temacie `/new_unique_topic_name`. W celu sprawdzenia poprawności działania Subscribiera, który będzie wyświetlał odebrane dane należy uruchomić utworzony wcześniej Publisher:

```
1 ros2 run example_tsr_project new_talker_node
```

Co powoduje wyświetlanie odbieranych danych w terminalu z uruchomionym węzłem z Subscriberem (`simple_subscriber`) (Rys. 5.2)



```
colcon build [1/1 done] [0 ongoing]
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 run example_tsr_project simple_subscriber
[INFO] [1740143468.367233875] [simple_subscriber]: Id wiadomości: 123
Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740143469.316131071] [simple_subscriber]: Id wiadomości: 123
Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740143470.316281831] [simple_subscriber]: Id wiadomości: 123
Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740143471.314003255] [simple_subscriber]: Id wiadomości: 123
Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740143472.316466324] [simple_subscriber]: Id wiadomości: 123
Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[INFO] [1740143473.316082506] [simple_subscriber]: Id wiadomości: 123
Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
```

Rys. 5.2: Wynik uruchomienia nowego węzła `simple_subscriber`

6 Procedura tworzenia Subscribiera dla istniejącego Publishera

6.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy Subscriber. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

6.2 Utworzenie pliku `new_subscriber_turtlesim_pose.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik *`new_subscriber_turtlesim_pose.py`* w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/new_subscriber_turtlesim_pose.py
2 chmod +x example_tsr_project/new_subscriber_turtlesim_pose.py
```

Nowo utworzony Subscriber powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/new_subscriber_turtlesim_pose.py
```

6.3 Sprawdzenie typu wiadomości wysyłanej przez istniejący Publisher.

W tym celu na początku należy uruchomić w nowym terminalu symulację dla robotów żółwi (`turtlesim`), jeśli nie jest uruchomiona:

```
1 ros2 run turtlesim turtlesim_node
```

Następnie w **nowym terminalu** wyświetlamy listę tematów obsługiwanych przez symulację:

```
1 ros2 topic list
```

W symulacji dla każdego robota dostępne są 3 tematy:

- `/turtle1/cmd_vel` - do wysyłania prędkości sterujących robotem.
- `/turtle1/color_sensor` - do odczytu aktualnego koloru na jakim znajduje się robot. Jeśli ma ustawiony pisak to jest to cały czas kolor pisaka.
- `/turtle1/pose` - do odczytu aktualnego położenia robota.

`turtle1` odnosi się do nazwy utworzonego robota, jeśli będzie ich więcej to tematy będą się między sobą różniły się tylko tą nazwą. Pozostała część pozostanie bez zmian. Ponadto każda wiadomość będzie tego samego typu. Różnica jest w przesyłanych danych (wiadomość jest uzupełniona innymi wartościami).

Dla tematu od pozycji `pose` dla węzła symulacji utworzony jest Publisher i dla niego zostanie napisany nowy węzeł (node) zawierający Subscriber odpowiedzialny za odbieranie aktualnej pozycji robota. W wyniku jego działania będzie wyświetlane dla robota aktualne położenie i orientacja przeliczona z radianów na stopnie.

Napisanie Subscribiera dla istniejącego Publishera wymaga znajomości tematu na jakim publikowane są wiadomości oraz ich typu. Znając już temat `/turtle1/pose`, na którym publikowane są wiadomości należy sprawdzić typ danych.

```
1 ros2 topic type /turtle1/pose
```

a następnie sprawdzić strukturę wiadomości, dla wyświetlonego typu:

```
1 ros2 interface show turtlesim/msg/Pose
```

Wynik działania kolejnych poleceń przedstawia (Rys. 6.1):



```
colcon build [1/1 done] [0 ongoing]
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 topic type /turtle1/pose
turtlesim/msg/Pose
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$
```

Rys. 6.1: Sprawdzenie typu i struktury wiadomości dla istniejącego Publishera dla tematu `/turtle1/pose`

6.4 Napisanie kodu dla węzła

Następnie należy utworzyć nowy węzeł z Subscriberem odbierającym na temacie `/turtle1/pose` wiadomość typu `Pose` z pakietu `turtlesim`.

Następnie należy napisać Subscribiera (zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu):

```
1 import rclpy # import biblioteki do obsługi ROS2
2 import math
3 from rclpy.node import Node
4 from turtlesim.msg import Pose # Import wiadomości ROS2 typu Pose z pakietu turtlesim
5
6 class TurtlePoseSubscriber(Node): # Utworzenie klasy dla Node'a
```

```
7 def __init__(self):
8     super().__init__('turtlesim_pose_subscriber') # unikalna nazwa node'a
9
10    # Utworzenie atrybutu klasy będącego Subscriberem wiadomości typu Pose na temacie /turtle1/pose.
11    self.subscription = self.create_subscription(Pose, '/turtle1/pose', self.callback, 10)
12
13    def callback(self, msg):
14        x = msg.x
15        y = msg.y
16        theta = math.degrees(msg.theta)
17        self.get_logger().info(f'{x=}, {y=}, {theta=} stopni')
18
19
20 def main(args=None):
21     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
22     rclpy.init(args=args)
23
24     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy TurtlePoseSubscriber
25     node = TurtlePoseSubscriber()
26
27     # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
28     #   ↪ zakończenie działania programu.
29     rclpy.spin(node)
30
31     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
32     node.destroy_node()
33
34     # Zamyka połączenie z ROS2.
35     rclpy.shutdown()
36
37 if __name__ == '__main__':
38     # uruchomienie node'a
39     main()
```

6.5 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za odbieranie utworzonej przed chwilą wiadomości. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```
1 entry_points={
2     'console_scripts': [
3         'turtle1_pose_sub = example_tsr_project.new_subscriber_turtlesim_pose:main',
4     ],
5 }
```

Lewa strona w linii z dodanym **turtle1_pose_sub** odnosi się do unikalnej nazwy, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa nie będzie widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`). Zamiast tego będzie widoczna nazwa wynikająca z konfiguracji węzła w pliku z programem. Wpisana w konstruktorze tam gdzie pojawia się `super().__init__('turtlesim_pose_subscriber')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `turtlesim_pose_subscriber`.

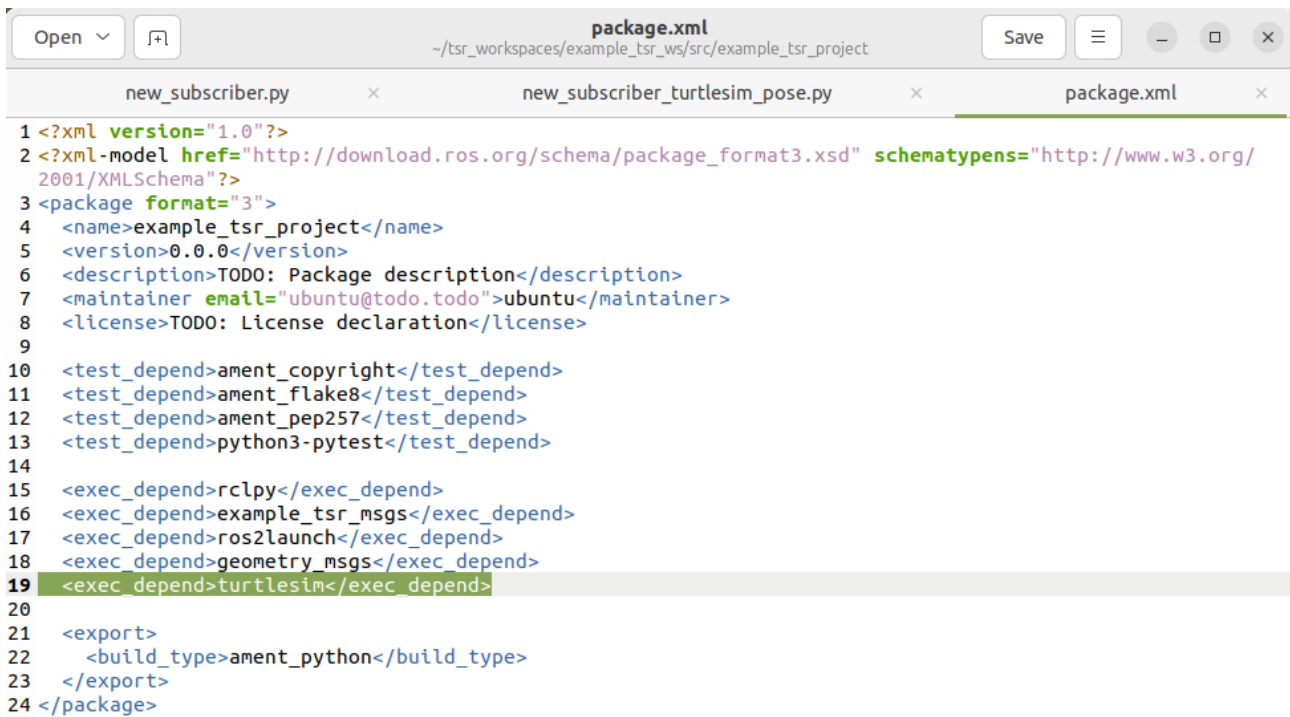
Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `new_subscriber_turtlesim_pose.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

6.6 Modyfikacja `package.xml` w pakiecie *example_tsr_project*.

Należy dodać następujące zależności do pliku `package.xml`:

```
1 <exec_depend>turtlesim</exec_depend>
```

Dodanie linii do pliku `package.xml` (Rys. 6.2):



Rys. 6.2: Plik package.xml po dodaniu zależności od turtlesim

6.7 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (example_tsr_ws):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny source w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

6.8 Weryfikacja działania

W terminalu należy wpisać:

```
1 ros2 run example_tsr_project turtle1_pose_sub
```

W wyniku uruchomienia węzła odbierana jest wiadomość na temacie /turtle1/pose typu Pose. W logach w terminalu powinno być widoczne aktualne położenie x, y oraz orientacja theta w stopniach. Ponadto w terminalu wyświetlane są odbierane wiadomości (Rys. 6.3).

```

File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 run example_tsr_project turtle1_pose_sub
[INFO] [1740146328.665927981] [turtle1_pose_subscriber]: x=6.104990482330322, y=5.721758365311035, theta=34.19004185287291 stopni
[INFO] [1740146328.611993032] [turtle1_pose_subscriber]: x=6.118079662322998, y=5.730959892272949, theta=35.10677276779909 stopni
[INFO] [1740146328.627712915] [turtle1_pose_subscriber]: x=6.1310200691223145, y=5.74036979675293, theta=36.023507097819866 stopni
[INFO] [1740146328.643463161] [turtle1_pose_subscriber]: x=6.143808364868164, y=5.749985218048096, theta=36.94023801274605 stopni
[INFO] [1740146328.659398636] [turtle1_pose_subscriber]: x=6.1564412117004395, y=5.7598042488098145, theta=37.856972342766824 stopni
[INFO] [1740146328.676651981] [turtle1_pose_subscriber]: x=6.168914794921875, y=5.769824504852295, theta=38.77370325769301 stopni
[INFO] [1740146328.691819585] [turtle1_pose_subscriber]: x=6.181227207183838, y=5.78004264831543, theta=39.69043758771378 stopni
[INFO] [1740146328.707723326] [turtle1_pose_subscriber]: x=6.193374156951904, y=5.790456295013428, theta=40.607168502639965 stopni
[INFO] [1740146328.724015326] [turtle1_pose_subscriber]: x=6.205353260040283, y=5.801063537597656, theta=41.52390283266074 stopni
[INFO] [1740146328.739773474] [turtle1_pose_subscriber]: x=6.217160701751709, y=5.81186056137085, theta=42.44063374758692 stopni
[INFO] [1740146328.755970070] [turtle1_pose_subscriber]: x=6.228794097900391, y=5.822845458984375, theta=43.357368077607696 stopni
[INFO] [1740146328.772226226] [turtle1_pose_subscriber]: x=6.240250110626221, y=5.834014892578125, theta=44.27409899253389 stopni
[INFO] [1740146328.790082051] [turtle1_pose_subscriber]: x=6.251526355743408, y=5.84536600112915, theta=45.19082990746007 stopni
[INFO] [1740146328.804590867] [turtle1_pose_subscriber]: x=6.2626198185546875, y=5.85689640045166, theta=46.107564237480844 stopni
[INFO] [1740146328.820244598] [turtle1_pose_subscriber]: x=6.273526191711426, y=5.868602752685547, theta=47.02429515240703 stopni
[INFO] [1740146328.836167676] [turtle1_pose_subscriber]: x=6.284244537353516, y=5.880482196807861, theta=47.9410294824278 stopni
[INFO] [1740146328.851851946] [turtle1_pose_subscriber]: x=6.294771194458008, y=5.892531394958496, theta=48.857760397353985 stopni
[INFO] [1740146328.870028069] [turtle1_pose_subscriber]: x=6.305103778839111, y=5.904747486114502, theta=49.77449472737476 stopni
[INFO] [1740146328.884132033] [turtle1_pose_subscriber]: x=6.315239906311035, y=5.9171271324157715, theta=50.69122564230094 stopni
[INFO] [1740146328.900126892] [turtle1_pose_subscriber]: x=6.325517671585083, y=5.929667949676514, theta=51.607959972321716 stopni

```

Rys. 6.3: Sprawdzenie poprawności działania węzła `turtlesim_pose_subscriber`

Po uruchomieniu ręcznego sterowania dla robota w nowym terminalu można zauważyć, że wartości dla pozycji zmieniają się.

```
1 ros2 run turtlesim turtle_teleop_key
```

7 Procedura tworzenia jednoczesnego Publisher'a i Subscriber'a

7.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony jednoczesny Publisher i Subscriber odpowiadający za ruch robota w prawo a później w lewo. Należy przejść do pakietu z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

7.2 Utworzenie pliku `pub_sub_turtlesim.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik `pub_sub_turtlesim.py` w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/pub_sub_turtlesim.py
2 chmod +x example_tsr_project/pub_sub_turtlesim.py
```

Nowo utworzony plik powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/pub_sub_turtlesim.py
```

7.3 Napisanie kodu dla węzła

Następnie należy utworzyć nowy węzeł odbierając wiadomość o pozycji pojawiającą się na temacie `/turtle1/pose` i na jej podstawie wysyłający odpowiednie prędkości sterujące na temacie `/turtle1/cmd_vel`. W tym celu scalone zostaną fragmenty kodu od Publishera i Subscribiera dla symulacji turtlesim.

Następnie należy uzupełnić plik (zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu):

```
1 import rclpy # import biblioteki do obsługi ROS2
2 from rclpy.node import Node
3 from turtlesim.msg import Pose # Import wiadomości ROS2 typu Pose z pakietu turtlesim
4 from geometry_msgs.msg import Twist # Import wiadomości ROS2 typu Twist z pakietu geometry_msgs
5
6 class ContinuousMovementTurtlesim(Node): # Utworzenie klasy dla Node'a
7     def __init__(self):
8         super().__init__('continuous_turtle_movement') # unikalna nazwa node'a
9         self.direction_right = True # Odpowiada za nadzorowanie kierunku ruchu robota True - w prawo, False
            ↳ - w lewo
10        # Utworzenie atrybutu klasy będącego Publisherem wiadomości typu Twist na temacie /turtle1/cmd_vel.
11        self.publisher_ = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
12
13        # Utworzenie atrybutu klasy będącego Subscribierem wiadomości typu Pose na temacie /turtle1/pose.
14        self.subscription = self.create_subscription(Pose, '/turtle1/pose', self.callback, 10)
15
16    def callback(self, pose_msg):
17        # Subscriber odbiera informację o położeniu i je analizuje. W funkcji pojawia się wystanie
            ↳ wiadomości
18        vel_msg = Twist()
19        if self.direction_right:
20            vel_msg.linear.x = 0.5
21            vel_msg.angular.z = 0.0
22        else:
23            vel_msg.linear.x = -0.5
24            vel_msg.angular.z = 0.0
25
26        if pose_msg.x > 7:
27            self.direction_right = False
28        elif pose_msg.x < 2:
29            self.direction_right = True
30
31        # wystanie przeanalizowanych danych
32        self.publisher_.publish(vel_msg)
33
34
35 def main(args=None):
36     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
37     rclpy.init(args=args)
38
39     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy ContinuousMovementTurtlesim
40     node = ContinuousMovementTurtlesim()
41
42     # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
            ↳ zakończenie działania programu.
43     rclpy.spin(node)
44
45     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
46     node.destroy_node()
47
48     # Zamyka połączenie z ROS2.
49     rclpy.shutdown()
50
51 if __name__ == '__main__':
52     # uruchomienie node'a
53     main()
```

7.4 Aktualizacja *setup.py*

W pakiecie `example_tsr_project` w pliku `setup.py`. Należy zmodyfikować linię z `entry_points` tak, aby dodać nowo utworzony węzeł.

```
1 entry_points={
2     'console_scripts': [
```

```
3     'turtle1_continuous_movement = example_tsr_project.pub_sub_turtlesim:main',  
4     ],  
5 }
```

7.5 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

7.6 Weryfikacja działania

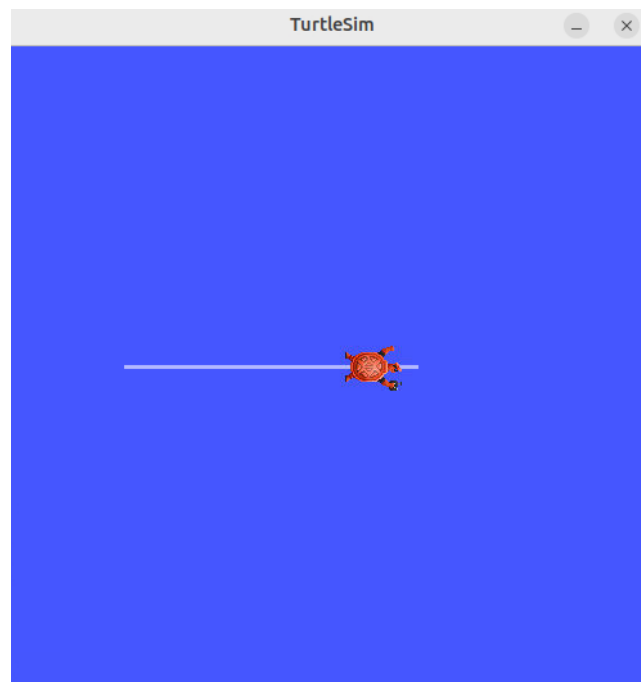
Uruchomienie czystej symulacji `turtlesim` bez dodatkowych węzłów odpowiedzialnych za wysyłanie prędkości dla `turtle1`.

```
1 ros2 run turtlesim turtlesim_node
```

W nowym terminalu należy uruchomić nowo utworzony węzeł:

```
1 ros2 run example_tsr_project turtle1_continuous_movement
```

Co powoduje poruszanie się robota w prawo i w lewo wraz z rysowaniem linii (Rys. 7.1)



Rys. 7.1: Wynik uruchomienia ciągłego ruchu robota

8 Dokumentacja i inne linki

Rozdział może zawierać dokumentację poleceń, linki do źródeł na podstawie, których powstała instrukcja oraz inne linki pozwalające poszerzyć wiedzę.

- Dokumentacja ROS Humble

<https://docs.ros.org/en/humble/index.html>

- Więcej o ROS2 Topic

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 topic --help
usage: ros2 topic [-h] [--include-hidden-topics]
               Call `ros2 topic <command> -h` for more detailed usage. ...

Various topic related sub-commands

options:
  -h, --help            show this help message and exit
  --include-hidden-topics
                        Consider hidden topics as well

Commands:
  bw      Display bandwidth used by topic
  delay   Display delay of topic from timestamp in header
  echo    Output messages from a topic
  find    Output a list of available topics of a given type
  hz      Print the average publishing rate to screen
  info    Print information about a topic
  list    Output a list of available topics
  pub     Publish a message to a topic
  type    Print a topic's type

Call `ros2 topic <command> -h` for more detailed usage.
```

Rys. 8.1: Dokumentacja polecenia `ros2 topic`

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface --help
usage: ros2 interface [-h]
               Call `ros2 interface <command> -h` for more detailed
               usage. ...

Show information about ROS interfaces

options:
  -h, --help            show this help message and exit

Commands:
  list      List all interface types available
  package   Output a list of available interface types within one package
  packages  Output a list of packages that provide interfaces
  proto     Output an interface prototype
  show      Output the interface definition

Call `ros2 interface <command> -h` for more detailed usage.
```

Rys. 8.2: Dokumentacja polecenia `ros2 interface`

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 run --help
usage: ros2 run [-h] [--prefix PREFIX] package_name executable_name ...

Run a package specific executable

positional arguments:
  package_name      Name of the ROS package
  executable_name   Name of the executable
  argv              Pass arbitrary arguments to the executable

options:
  -h, --help          show this help message and exit
  --prefix PREFIX     Prefix command, which should go before the executable.
                     Command must be wrapped in quotes if it contains spaces
                     (e.g. --prefix 'gdb -ex run --args').
```

Rys. 8.3: Dokumentacja polecenia `ros2 run`